

DevOps Handbook

1 DevOps Handbook — Master Table of Contents

- [1.1 How to read this handbook](#)
- [1.2 Part I — Introduction & Mindset \(Pages 1–60\)](#)
- [1.3 Part II — Linux & Shell Mastery \(Pages 61–140\)](#)
- [1.4 Part III — Git & Collaboration \(Pages 141–200\)](#)
- [1.5 Part IV — Networking for DevOps \(Pages 201–280\)](#)
- [1.6 Part V — Scripting & Programming \(Pages 281–360\)](#)
- [1.7 Part VI — Infrastructure as Code \(Pages 361–480\)](#)
- [1.8 Part VII — Containers \(Pages 481–580\)](#)
- [1.9 Part VIII — Kubernetes \(Pages 581–720\)](#)
- [1.10 Part IX — CI/CD \(Pages 721–840\)](#)
- [1.11 Part X — Observability & SRE \(Pages 841–960\)](#)
- [1.12 Part XI — Security \(DevSecOps\) \(Pages 961–1080\)](#)
- [1.13 Part XII — Advanced Topics \(Pages 1081–1200\)](#)
- [1.14 Expansion status](#)

2 Chapter 1: What Is DevOps? History and Culture

- [2.1 1.1 The problem DevOps solves](#)
- [2.2 1.2 A working definition](#)
- [2.3 1.3 Brief history](#)
- [2.4 1.4 Culture before tools](#)
- [2.5 1.5 DevOps practices \(overview\)](#)
- [2.6 1.6 Organizational models](#)
 - [2.6.1 Functional silos \(anti-pattern for DevOps\)](#)
 - [2.6.2 Cross-functional product teams \(recommended\)](#)
 - [2.6.3 DevOps enablement / platform team](#)
- [2.7 1.7 What DevOps is NOT](#)
- [2.8 1.8 Your learning path in this handbook](#)
- [2.9 1.9 Chapter summary](#)
- [2.10 Lab 1.1 — Reflect on your current workflow](#)
- [2.11 Review questions](#)

3 Chapter 2: CALMS, Three Ways, and Team Topologies

- [3.1 2.1 The CALMS framework](#)
 - [3.1.1 Culture \(C\)](#)
 - [3.1.2 Automation \(A\)](#)
 - [3.1.3 Lean \(L\)](#)
 - [3.1.4 Measurement \(M\)](#)
 - [3.1.5 Sharing \(S\)](#)
- [3.2 2.2 The Three Ways \(Gene Kim\)](#)
 - [3.2.1 First Way: Flow \(left to right\)](#)
 - [3.2.2 Second Way: Feedback \(right to left\)](#)
 - [3.2.3 Third Way: Continuous learning and experimentation](#)
- [3.3 2.3 Team Topologies \(Skelton & Pais\)](#)
 - [3.3.1 Interaction modes](#)
- [3.4 2.4 Conway's Law and reverse Conway maneuver](#)
- [3.5 2.5 Maturity model \(informal\)](#)
- [3.6 2.6 Chapter summary](#)
- [3.7 Lab 2.1 — CALMS assessment](#)
- [3.8 Review questions](#)

4 Chapter 5: Linux Fundamentals — Filesystem, Users, Permissions

- [4.1 5.1 Why Linux dominates DevOps](#)
- [4.2 5.2 Filesystem hierarchy \(FHS\)](#)
- [4.3 5.3 Essential navigation commands](#)
- [4.4 5.4 File operations](#)

- [4.4.1 Finding files](#)
- [4.5 5.5 Users, groups, and permissions](#)
 - [4.5.1 Users and groups](#)
 - [4.5.2 Permission model](#)
 - [4.5.3 Special permissions](#)
- [4.6 5.6 Package management \(Debian/Ubuntu\)](#)
- [4.7 5.7 Text processing \(daily DevOps tools\)](#)
- [4.8 5.8 Environment and shell](#)
- [4.9 5.9 SSH — remote access](#)
- [4.10 5.10 Chapter summary](#)
- [4.11 Lab 5.1 — Linux sandbox](#)
- [4.12 Review questions](#)

1 DevOps Handbook — Master Table of Contents

Target length: ~1,200 pages (~480,000 words)
Audience: Beginner → Advanced
Format: Each numbered section ≈ one “page” of ~400 words in the final edition

1.1 How to read this handbook

Work through **Parts I-XII** in order. Labs (marked) add hands-on time beyond page counts. Cross-reference the [AWS Handbook](#) for cloud-specific depth.

1.2 Part I — Introduction & Mindset (Pages 1-60)

Ch	Title	Pages
1	What Is DevOps? History and Culture	1-15
2	CALMS, Three Ways, and Team Topologies	16-30
3	DevOps vs SRE vs Platform Engineering	31-42
4	Career Paths, Certifications, and Learning Strategy	43-60

Written: [Chapter 1](#), [Chapter 2](#)

1.3 Part II — Linux & Shell Mastery (Pages 61-140)

Ch	Title	Pages
5	Linux Fundamentals: Filesystem, Users, Permissions	61-85
6	Processes, systemd, and Service Management	86-105
7	Networking on Linux: ip, ss, curl, dig	106-120
8	Bash Scripting for Automation	121-140

Written: [Chapter 5](#)

1.4 Part III — Git & Collaboration (Pages 141-200)

Ch	Title	Pages
9	Git Internals: Objects, Refs, and the DAG	141-158
10	Branching Strategies and Code Review	159-175
11	GitHub/GitLab Workflows: PRs, Issues, Actions intro	176-200

1.5 Part IV — Networking for DevOps (Pages 201-280)

Ch	Title	Pages
12	OSI Model, TCP/IP, DNS Deep Dive	201-225
13	HTTP/HTTPS, TLS, and Certificates	226-245
14	Load Balancing, Proxies, and CDN Concepts	246-265
15	Cloud Networking Patterns (VPC overview)	266-280

1.6 Part V — Scripting & Programming (Pages 281-360)

Ch	Title	Pages
16	Python for DevOps: boto3, requests, pathlib	281-305
17	Go for CLI Tools and Operators	306-325
18	JSON/YAML, Jinja2, and Templating	326-345
19	Testing Automation Scripts	346-360

1.7 Part VI — Infrastructure as Code (Pages 361-480)

Ch	Title	Pages
20	IaC Principles: State, Idempotency, Drift	361-380
21	Terraform: HCL, Providers, Modules	381-410
22	Terraform: Workspaces, Backends, CI Integration	411-430
23	Ansible: Playbooks, Roles, Inventories	431-450
24	Packer & Golden Images	451-465
25	Policy as Code: OPA, Checkov, Sentinel	466-480

1.8 Part VII — Containers (Pages 481-580)

Ch	Title	Pages
26	Docker: Images, Dockerfile, Multi-stage Builds	481-505
27	Container Networking, Volumes, and Security	506-525
28	Docker Compose and Local Dev Environments	526-540
29	Container Registries and Image Signing	541-560
30	Rootless Containers and Alternatives (Podman)	561-580

1.9 Part VIII — Kubernetes (Pages 581-720)

Ch	Title	Pages
31	K8s Architecture: Control Plane & Node Components	581–605
32	Workloads: Pod, Deployment, StatefulSet, DaemonSet	606–630
33	Services, Ingress, and Network Policies	631–655
34	ConfigMaps, Secrets, and Storage (PV/PVC)	656–675
35	Helm, Kustomize, and GitOps (Argo CD)	676–700
36	RBAC, Pod Security, and Hardening	701–720

1.10 Part IX — CI/CD (Pages 721–840)

Ch	Title	Pages
37	CI/CD Concepts and Pipeline Design	721–745
38	GitHub Actions: Workflows, Secrets, Runners	746–770
39	Jenkins: Pipelines as Code (Declarative)	771–790
40	GitLab CI and Multi-platform Comparison	791–810
41	Deployment Strategies: Blue/Green, Canary, Feature Flags	811–825
42	DORA Metrics and Continuous Improvement	826–840

1.11 Part X — Observability & SRE (Pages 841–960)

Ch	Title	Pages
43	Monitoring Fundamentals: Metrics, Logs, Traces	841–865
44	Prometheus & Grafana Stack	866–890
45	ELK/EFK and Log Aggregation	891–910
46	OpenTelemetry and Distributed Tracing	911–930
47	SLOs, Error Budgets, and Incident Response	931–945
48	Chaos Engineering and Game Days	946–960

1.12 Part XI — Security (DevSecOps) (Pages 961–1080)

Ch	Title	Pages
49	Shift-Left Security in the Pipeline	961–980
50	SAST, DAST, SCA, and Container Scanning	981–1000
51	Secrets Management: Vault, SOPS, External Secrets	1001–1020

52	Supply Chain: SBOM, Sigstore, SLSA	1021-1040
53	Runtime Security: Falco, OPA Gatekeeper	1041-1060
54	Compliance Automation and Audit Trails	1061-1080

1.13 Part XII — Advanced Topics (Pages 1081-1200)

Ch	Title	Pages
55	Platform Engineering & Internal Developer Platforms	1081-1100
56	Multi-Cloud and Hybrid Patterns	1101-1115
57	FinOps and Cost-Aware Engineering	1116-1130
58	MLOps Overview for DevOps Engineers	1131-1145
59	Capstone: End-to-End Production System Design	1146-1180
60	Appendix: Command Cheatsheets, Glossary, Index	1181-1200

1.14 Expansion status

Part	Status
I	Ch 1-2 written (~30 pages)
II	Ch 5 written (~25 pages)
III-XII	Outlined; request by chapter number to generate

To request a chapter: *“Expand DevOps Handbook Chapter 21 (Terraform modules)”*

2 Chapter 1: What Is DevOps? History and Culture

DevOps Handbook — Part I, Pages 1–15

2.1 1.1 The problem DevOps solves

Before DevOps, software organizations often split into two camps:

- **Development** — wrote features quickly, measured by release velocity.
- **Operations** — kept systems stable, measured by uptime and incident count.

These goals frequently conflicted. Developers threw code “over the wall”; operators resisted change because every deployment risked an outage. Releases were rare, painful, and manual. When something broke in production, blame flowed in both directions.

DevOps is not a single tool or job title. It is a **cultural and technical movement** that unifies development and operations around shared goals: deliver value to users **fast** and **reliably**.

2.2 1.2 A working definition

***DevOps** is the practice of aligning people, processes, and technology so that software can be built, tested, released, and operated continuously—with feedback loops that shorten the time from idea to production.*

Key outcomes:

Outcome	What it means
Shorter lead time	Hours or days from commit to production, not months
Higher deployment frequency	Many small releases instead of rare big-bang releases
Lower change failure rate	Most changes succeed without rollback
Faster recovery	When failures happen, restore service quickly

These four metrics are known as **DORA metrics** (DevOps Research and Assessment) and are the standard way to measure DevOps maturity.

2.3 1.3 Brief history

Year	Milestone
2007–2009	Flickr, Etsy, and others share “10+ deploys per day” stories
2009	First DevOpsDays conference in Ghent, Belgium (Patrick Debois)
2010	“The Phoenix Project” novel popularizes DevOps narrative
2013	“The DevOps Handbook” (Gene Kim et al.) codifies practices
2016+	Cloud-native, containers, and Kubernetes accelerate adoption
2020s	Platform engineering, GitOps, and AI-assisted ops emerge

The term **DevOps** blends **Development** + **Operations**. It intentionally breaks down silos rather than creating a third silo called “DevOps team” that sits between dev and ops.

2.4 1.4 Culture before tools

A common mistake is buying Jenkins, Kubernetes, or Terraform and calling it “DevOps.” Tools enable DevOps; they do not create it.

Cultural pillars:

1. **Shared ownership** — The team that builds a service also runs it (“you build it, you run it”).
 2. **Blameless postmortems** — Focus on systems and processes, not individuals, after incidents.
 3. **Automation of toil** — Repetitive manual work is eliminated or codified.
 4. **Continuous learning** — Experimentation, feedback, and improvement are expected.
 5. **Customer focus** — Every decision ties back to user value.
-

2.5 1.5 DevOps practices (overview)

You will study each in depth later in this handbook:

Practice	Summary
Version control	Everything in Git: code, config, IaC, docs
CI/CD	Automated build, test, deploy pipelines
Infrastructure as Code	Reproducible environments (Terraform, Ansible)
Monitoring & observability	Know what production is doing in real time
Microservices & containers	Smaller deployable units (optional but common)
Security integration (DevSecOps)	Security checks in the pipeline, not at the end

2.6 1.6 Organizational models

2.6.1 Functional silos (anti-pattern for DevOps)

Separate dev, QA, ops, and security teams with handoffs.

2.6.2 Cross-functional product teams (recommended)

One team owns a product or service end-to-end: code, infra, on-call, roadmap.

2.6.3 DevOps enablement / platform team

A **platform team** builds internal tools (CI templates, K8s clusters, golden paths) so product teams move faster. This is **not** a ticket-taking ops team—it is a **product for developers**.

2.7 1.7 What DevOps is NOT

- **Not “NoOps”** — Operations expertise is more important at scale, not less.
- **Not only for startups** — Enterprises adopt DevOps for compliance and speed.

- **Not replacing developers with operators** — It merges responsibilities skillfully.
- **Not a certification checkbox** — Culture and practice matter more than badges.

2.8 1.8 Your learning path in this handbook

[illegible]

Invest 70% of study time in **hands-on labs**. Reading without doing builds false confidence.

2.9 1.9 Chapter summary

- DevOps unifies dev and ops around **speed** and **reliability**.
- **Culture** (shared ownership, blameless learning) precedes tools.
- Measure progress with **DORA metrics**.
- Cross-functional teams and platform engineering are modern expressions of DevOps.

2.10 Lab 1.1 — Reflect on your current workflow

1. Map your (or a sample) team's path from code commit to production.
2. Count manual steps and handoffs.
3. Identify one bottleneck and one automation opportunity.
4. Write a one-paragraph "DevOps vision" for that team.

2.11 Review questions

1. Name the four DORA metrics.
2. Why is “DevOps team” as a silo problematic?
3. What year and event is credited with coining “DevOps”?
4. Give one cultural and one technical practice of DevOps.

Next: Chapter 2 — CALMS, Three Ways, and Team Topologies

3 Chapter 2: CALMS, Three Ways, and Team Topologies

DevOps Handbook — Part I, Pages 16–30

3.1 2.1 The CALMS framework

CALMS is a mnemonic for assessing DevOps maturity across five dimensions:

Letter	Dimension	Description
C	Culture	Collaboration, trust, shared goals
A	Automation	Eliminate manual, error-prone work
L	Lean	Reduce waste, small batches, flow
M	Measurement	Data-driven decisions, DORA metrics
S	Sharing	Knowledge, tools, failures, successes

3.1.1 Culture (C)

Healthy DevOps culture means developers understand production constraints and operators understand business priorities. Psychological safety allows people to report problems early.

3.1.2 Automation (A)

Automate builds, tests, deployments, infrastructure provisioning, and compliance checks. Manual runbooks become code. **Toil**—repetitive operational work that scales linearly with service growth—should be targeted for elimination.

3.1.3 Lean (L)

Inspired by Toyota Production System: optimize **flow**, limit work in progress (WIP), and deliver **small batches**. Large releases hide defects; small releases surface them quickly.

3.1.4 Measurement (M)

You cannot improve what you do not measure. Track deployment frequency, lead time, MTTR, change failure rate, plus business KPIs (revenue, user satisfaction).

3.1.5 Sharing (S)

Internal conferences, shared Slack channels, communal dashboards, and open postmortems spread expertise. Tools and patterns are reused, not reinvented per team.

3.2 2.2 The Three Ways (Gene Kim)

From *The Phoenix Project* and *The DevOps Handbook*, the **Three Ways** describe the principles of DevOps flow:

3.2.1 First Way: Flow (left to right)

Accelerate delivery from **development** → **operations** → **customer**.

- Make work visible (Kanban boards, pipeline status).
- Reduce batch sizes.
- Build quality in (automated tests at every stage).
- Never pass defects downstream.

Example: A commit triggers CI; only green builds reach staging; only approved artifacts reach production.

3.2.2 Second Way: Feedback (right to left)

Amplify feedback from **production** → **development**.

- Monitor user experience and system health.
- Swarm on problems; shorten feedback loops.
- Developers get production access (with guardrails).

Example: Error rate spike in Grafana pages the on-call engineer; linked traces show the bad deploy; rollback is automated.

3.2.3 Third Way: Continuous learning and experimentation

Create a culture of **improvement** and **risk-taking**.

- Blameless postmortems.
- Reserve time for improvement (20% ops time, hack days).
- Rehearse failures (game days, chaos engineering).

3.3 2.3 Team Topologies (Skelton & Pais)

Modern DevOps organizations structure teams intentionally:

Team type	Purpose
Stream-aligned	Delivers product features; primary value stream
Platform	Provides self-service infra (K8s, CI, databases)
Enabling	Coaches stream teams on new tech (temporary)
Complicated-subsystem	Owns deep specialty (e.g., video encoding, ML)

3.3.1 Interaction modes

- **Collaboration** — Two teams work closely for discovery.
- **X-as-a-Service** — Platform provides API/portal; stream teams consume.
- **Facilitating** — Enabling team mentors others.

Anti-pattern: One ops team serves 50 dev teams via tickets.

Better: Platform team offers paved roads; stream teams deploy autonomously.

3.4 2.4 Conway's Law and reverse Conway maneuver

“Organizations design systems that mirror their communication structure.” — Melvin Conway

If dev and ops are separate departments, you will get separate deployment artifacts and fragile handoffs.

Reverse Conway maneuver: Structure teams the way you want the architecture to look. Want autonomous microservices? Give teams end-to-end ownership of each service.

3.5 2.5 Maturity model (informal)

Level	Characteristics
1 — Initial	Manual deploys, silos, hero culture
2 — Managed	Some CI, version control, basic monitoring
3 — Defined	IaC, automated testing, standardized pipelines
4 — Measured	SLOs, DORA tracking, blameless postmortems
5 — Optimizing	Continuous experimentation, platform self-service

Use this for self-assessment, not as a rigid certification ladder.

3.6 2.6 Chapter summary

- **CALMS** evaluates Culture, Automation, Lean, Measurement, Sharing.
 - The **Three Ways** optimize flow, feedback, and learning.
 - **Team Topologies** align org structure with delivery goals.
 - Apply **Conway's Law** consciously when designing teams.
-

3.7 Lab 2.1 — CALMS assessment

Score your team 1–5 on each CALMS letter. Pick the lowest score and draft one actionable improvement per dimension.

3.8 Review questions

1. What does the “L” in CALMS stand for, and what Lean concept supports it?
 2. Describe the First Way in one sentence.
 3. Name the four team types in Team Topologies.
 4. What is a reverse Conway maneuver?
-

Next: Part II — [Chapter 5: Linux Fundamentals](#)

4 Chapter 5: Linux Fundamentals — Filesystem, Users, Permissions

DevOps Handbook — Part II, Pages 61–85

4.1 5.1 Why Linux dominates DevOps

Most servers, containers, and cloud instances run **Linux**. As a DevOps engineer you will SSH into machines, debug production issues, write shell scripts, and configure services daily. Windows matters too (Active Directory, .NET shops), but Linux fluency is non-negotiable.

Common distributions:

Distro	Typical use
Ubuntu	Cloud default, tutorials, desktops
Debian	Stable base for Ubuntu and containers
RHEL / Rocky / Alma	Enterprise, regulated industries
Amazon Linux	AWS-optimized

4.2 5.2 Filesystem hierarchy (FHS)

Path	Purpose
/	Root of entire tree
/home	User home directories (/home/alice)
/root	root user's home
/etc	Configuration files
/var	Variable data: logs (/var/log), caches
/tmp	Temporary files (often cleared on reboot)
/usr	User programs and libraries
/bin , /sbin	Essential binaries
/opt	Optional third-party software
/proc , /sys	Virtual filesystems for kernel info

Everything is a **file** in Unix—including devices and sockets.

4.3 5.3 Essential navigation commands

```
pwd           # Print working directory
ls -lah       # List files (human-readable, including hidden)
cd /etc/nginx # Change directory
cd ..         # Parent directory
cd ~          # Home directory
```

```
tree -L 2 /etc      # Tree view (install: apt install tree)
```

```
cp source dest           # Copy
cp -r dir1 dir2          # Copy directory recursively
mv old new               # Move or rename
rm file                  # Delete (careful!)
rm -rf directory         # Delete directory recursively (very careful!)
mkdir -p path/to/dir     # Create nested directories
touch file.txt           # Create empty file or update timestamp
cat file                 # Print file contents
less /var/log/syslog     # Paginated view (q to quit)
head -n 20 file          # First 20 lines
tail -f /var/log/app.log # Follow log in real time
```

```
find /var/log -name "*.log" -mtime -7 # Logs modified in 7 days
find / -type f -size +100M 2>/dev/null # Files larger than 100MB
locate nginx.conf # Fast index search (updatedb)
```

```
id                # Current user UID, GID, groups
whoami
cat /etc/passwd   # User accounts
cat /etc/group     # Groups
sudo command      # Run as root (if permitted)
su - username      # Switch user
```

Each file has three permission sets for **user (u)**, **group (g)**, **others (o)**:

```
-rwxr-xr-- 1 alice developers 4096 Jan 15 10:00 deploy.sh
|||||
|||  others: read
||   group: read, execute
|    user: read, write, execute
```

```
chmod 755 script.sh      # rwXR-XR-X
chmod u+x script.sh      # Add execute for user
chown alice:developers file
chgrp developers file
```

4.5.3 Special permissions

- **setuid** — Run as file owner (e.g., `passwd`)
- **setgid** — Run as group; dirs inherit group
- **sticky bit** — On `/tmp` , only owner can delete own files

4.6 5.6 Package management (Debian/Ubuntu)

```
sudo apt update           # Refresh package index
sudo apt upgrade -y       # Upgrade installed packages
sudo apt install nginx -y # Install package
apt search nginx          # Search
apt show nginx            # Package info
sudo apt remove nginx     # Remove
dpkg -l | grep nginx     # List installed
```

RHEL/Rocky equivalent: `dnf install` , `dnf update` .

4.7 5.7 Text processing (daily DevOps tools)

```
grep -r "ERROR" /var/log/app/ # Search recursively
grep -E "timeout|refused" log.txt # Extended regex
awk '{print $1}' access.log    # Column extraction
sed 's/old/new/g' file.txt    # Stream editor
sort file | uniq -c | sort -rn | head # Count unique lines
wc -l file                    # Line count
```

Pipes (|) chain commands: stdout of one → stdin of next.

4.8 5.8 Environment and shell

```
echo $PATH                # Executable search path
export MY_VAR=hello       # Set environment variable
env                        # All environment variables
which python3              # Path to executable
alias ll='ls -lah'        # Command shortcut
```

Shell config files: `~/.bashrc` , `~/.profile` (sourced on login).

4.9 5.9 SSH — remote access

```
ssh user@hostname
ssh -i ~/.ssh/mykey.pem ec2-user@1.2.3.4
scp file user@host:/remote/path
rsync -avz ./local/ user@host:/remote/ # Efficient sync
```

`~/.ssh/config` simplifies connections:

```
Host prod-web
  HostName 10.0.1.50
  User ubuntu
  IdentityFile ~/.ssh/prod.pem
```

Then: `ssh prod-web`

4.10 5.10 Chapter summary

- Linux FHS organizes `/etc`, `/var`, `/home`, and more.
 - Master `ls`, `cd`, `cp`, `mv`, `rm`, `find`, `grep`, `tail -f`.
 - Permissions: `user/group/other` + `chmod` / `chown`.
 - Use `apt` / `dnf` for packages; SSH for remote ops.
-

4.11 Lab 5.1 — Linux sandbox

1. Create user `devops-lab` with home directory.
 2. Create `~/scripts/deploy.sh` with `#!/bin/bash` and `echo "Deploying..."`.
 3. Set permissions to `755`; run it.
 4. Write a one-liner to count ERROR lines in a sample log file.
 5. Configure SSH key auth to a local VM or cloud instance.
-

4.12 Review questions

1. What directory holds system configuration files?
 2. What does permission `644` mean in symbolic notation?
 3. How do you follow a log file in real time?
 4. What is the difference between `cp` and `mv`?
-

Continue: Chapter 6 — Processes, systemd, and Service Management (outline in TOC)